

 DAY 30:
JAVASCRIPT
MODULES (ES6
IMPORT/EXPORT)



Lakshmiprasanna Vakada

Modern JavaScript (ES6+) introduced modules, allowing us to split code into separate files and reuse it efficiently. This improves code maintainability, reusability, and organization in large projects.

◆ Why Use Modules?

- ✓ Encapsulation: Avoids polluting the global scope
- ✓ Reusability: Write code once and import it anywhere
- ✓ Maintainability: Keeps code organized and readable
- ✓ Performance: Loads only the required code when needed

◆ Exporting in JavaScript Modules

We can export functions, variables, or classes from one file to another. There are two types of exports:

1 Named Exports (export)

Allows exporting multiple values from a module.

 Example: Exporting multiple values (in mathUtils.js)

```
// Exporting multiple functions
export function add(a, b) {
  return a + b;
}
export function subtract(a, b) {
  return a - b;
}
export const PI = 3.14159;
```

Importing named exports (in main.js)

```
import { add, subtract, PI } from
"./mathUtils.js";

console.log(add(5, 3)); // 8
console.log(subtract(10, 4)); // 6
console.log(PI); // 3.14159
```

2 Default Exports (export default)

Each module can have only one default export.

Example: Default export (in logger.js)

```
export default function logMessage(message) {
  console.log("Log:", message);
}
```



Importing a default export (in main.js)

```
import logMessage from "./logger.js";  
  
logMessage("Hello, Modules!"); // Log:  
Hello, Modules!
```

✓ Default exports can be renamed when importing:

```
import myLogger from "./logger.js";  
  
myLogger("Custom name for the function!");
```

◆ Mixing Named and Default Exports

You can combine both types of exports in a module.

📌 Example: Combined Exports (in utils.js)

```
export default function greet(name) {  
  return `Hello, ${name}!`;  
}  
  
export const version = "1.0";  
export function sayGoodbye() {  
  return "Goodbye!";  
}
```

Importing mixed exports (in main.js)

```
import greet, { version, sayGoodbye } from  
"./utils.js";
```

```
console.log(greet("Alice")); // Hello,  
Alice!
```

```
console.log(version); // 1.0
```

```
console.log(sayGoodbye()); // Goodbye!
```

◆ Dynamic Imports (import())

We can dynamically import modules when needed using `import()`. This is useful for lazy loading and improving performance.



Example: Dynamic Import

```
async function loadLogger() {  
  const module = await  
import("./logger.js");  
  module.default("This is a dynamically  
imported module!");  
}  
  
loadLogger();
```



Advantages of dynamic imports:

- Loads the module only when needed (better performance)
- Useful for code-splitting in large applications

◆ Module Scope & Strict Mode

- Modules are always in strict mode ("use strict" is enabled by default)
- Variables/functions inside a module are scoped to that module

◆ How to Use Modules in HTML?

To use modules in the browser, add the type="module" attribute in the <script> tag.

Example: Using modules in HTML

```
<script type="module" src="main.js">  
</script>
```

✓ Key Points for Browser Modules:

- Modules run in strict mode automatically
- They are deferred by default, meaning they don't block page loading
- CORS restrictions apply, so you must serve modules from an HTTP server